

Context Contagion in Real Transformers: A Progress Report

Dan Shumow*

June 2026 — *draft, in progress*

Abstract

Two companion analyses [1, 2] establish, *exactly* on a toy global-plus-cache Markov model, that in-context exploitability tracks *trust* rather than usefulness, and that self-reproducing token strings (*quines*) can be planted in a cache, hijack an occupied one, and spread between caches as a context-window worm—all cheapest and most transmissible under a longest-match (*induction-head*) surrogate. Those results were stated as *conjectures* for real transformers. This report is the running status of testing them on real models. So far we have built the harness and run the first two stages of the keystone experiment on GPT-2-small. We find (i) a clear *condensation knee*: a repeated out-of-distribution string begins to be regenerated past a sharp threshold in the repetition count, with the threshold *decreasing* in string length (the induction-like, length-robust signature, opposite of fixed-order brittleness); and (ii) that reproduction is predominantly *cue-dependent*—a non-matching prime mostly fails to elicit the string while a matching one reliably does—a behavioral induction signature obtained without interpretability tooling; and (iii) confirmed *mechanistically* that the model’s own induction heads carry it—ablating the canonical GPT-2 induction heads collapses reproduction near the knee, far beyond random-head damage. Controls further show the knee is a repetition (not context-length) effect though distance-sensitive, and the phenomena replicate in a second model family (Pythia). We then confirm the three downstream claims: (iv) poisonability tracks *trust*, not usefulness—repetitive gibberish is fully poisonable while useful non-repetitive prose is not ($\text{corr}(P, \text{useful}) = -0.20$ vs. $\text{corr}(P, \text{trust}) = +0.54$, C4); (v) a payload planted in a document hijacks generation only when recent *and* sufficiently repeated—delivery factors as $\text{entry} \times \text{lock-in}$ (C5); and (vi) output fed back as another context propagates as a *worm* with a critical string length (short strings reach an endemic load, long ones die out) and a *cross-tokenizer firebreak* (GPT-2→Pythia transmission collapses, C6). A *paired detector* closes the loop: scoring contexts by induction-trust on repeated spans separates poisoned from clean documents perfectly (AUC 1.0) and flags the poisonable configuration before it crosses the hijack knee—the transformer realization of the toy’s *bound-trust* defense. What remains is scale: a genuine size sweep and instruction-tuned models are capped by the 8 GB development machine. This document will be revised as those land.

1 Scope and status

This is a *progress report*, not a finished paper: sections marked **[TBD]** are planned but not yet run. The conceptual framing, the exact toy results, and the falsifiable conjecture all live in the companion write-ups [1, 2]; we recap only what is needed. The threat is concrete—context poisoning and

*Work-in-progress write-up of the open-source project **transformer-context-contagion**, the empirical follow-on to the toy-model analyses **trust-is-the-attack-surface**. All numbers and figures are reproduced by the **experiments/** scripts in that repository; the running log is **results_log.md**.

self-replicating prompts already target real retrieval-augmented and agent pipelines [6, 5]—and the toy substrate is a from-scratch analogue of the parametric-versus-in-context split studied by Bietti et al. [4]. The detailed experiment plan (claims, controls, statistics, phasing) is the repository’s `transformer_validation_plan.md` and `e1_plan.md`; this report records what those plans have produced so far.

Stage	Status	What
E1.0 smoke test	done	knee exists on GPT-2 (qualitative).
E1.1 measured knee	done	knee vs length with controls + CIs (§4).
E1.2 copy vs. frequency	done	cue-dependence; behavioral induction (§5).
E1.5 context confound	done	knee is an N effect; distance-sensitive (§6).
E1.3 size sweep	partial	cross-family replication; 8 GB caps it at $\leq 162\text{M}$ (§7).
E2 induction attribution	done	induction heads <i>causally</i> carry it (§8).
E4-lite trust, not usefulness	done	poisonability $\perp$ usefulness, $\parallel$ trust (§9).
E5-lite delivery in a document	done	hijack = entry $\times$ lock-in (§10).
E6 the worm / R_0	done	critical length; cross-tokenizer firebreak (§11).
E7 paired detector	done	flags poisonable repeated spans, AUC 1.0 (§12).
E1.4 modern / chat models	[TBD]	base vs. instruct (needs $>8\text{GB}$).
E1.3 size sweep ($\geq 355\text{M}$)	[TBD]	how the knee moves with scale (needs $>8\text{GB}$).

Table 1: Status of the experimental program. Stage numbering follows `transformer_validation_plan.md` and `e1_plan.md`.

2 Claims under test

Each maps a toy object to a transformer-measurable quantity; see the plan for the full list and the experiments E1–E6.

Toy result	Real-transformer prediction
C1 condensation knee	a repeated OOD string is regenerated past a sharp threshold in N
C2 induction is the engine	reproduction is carried by induction heads (copy-on-match)
C3 length penalty vanishes under induction	knee location roughly flat or <i>decreasing</i> in length
C4 exploitability tracks trust, not usefulness	repetitive contexts most poisonable, $\approx$ independent of benefit
C5 delivery = entry $\times$ lock-in	a planted span hijacks generation in a long context
C6 worm with $R_0 \approx T/(p k_{\text{thresh}})$	output fed back propagates; short strings spread, long die

Table 2: The conjecture, claim by claim. All six now have a real-transformer result on GPT-2-small: **C1**, **C3** (§4); **C2** behaviorally (§5) and mechanistically (§8); **C4** (§9); **C5** (§10); **C6** (§11). What remains is scale (size sweep, instruct models).

3 Method

Payloads. A *payload* is a short cyclic string S of token ids the model would essentially never emit on its own, so any reproduction is the copy mechanism, not the model’s taste—the analogue of the toy’s rainbow / de Bruijn quines. Payloads are built at the *token* level (length p is in tokens) from a pool of rare, round-trip-stable single tokens, as distinct-token (“rainbow”) cycles. We use R independent random payloads per length and average, with Wilson confidence intervals.

Scoring. Reproduction is scored at the level of *transitions*, not positions: a single slip only phase-shifts a positional comparison and tanks all later positions even while the model is faithfully reproducing. Given S and a generated continuation, the per-step fidelity is the fraction of steps obeying S ’s successor rule given the realized window; a continuation *reproduces* S if that fidelity exceeds $\frac{1}{2}$. $P(\text{reproduce})$ is the fraction of sampled continuations that do.

Controls. Three controls separate genuine pattern-copying from artifacts: (i) a *no-payload* ($N=0$) control—prime at the same start token but never show the payload—measures spontaneous emission, which must be ≈ 0 ; (ii) a *scrambled* control—the same tokens in shuffled order, destroying the repeated n -gram while preserving token frequencies; (iii) a *cue* control (§5) contrasting a matching with a non-matching generation prompt.

Setup. Results below are GPT-2-small (124M) [7], on an Apple-Silicon laptop (MPS/Metal backend), `torch 2.8`, `transformers 4.57`, sampled at temperature 1.0. This is a development-scale environment: it establishes the phenomena on one small model; the size sweep (E1.3) needs more compute.

4 E1.1: the condensation knee

We sweep the repetition count N and the string length p , estimating $P(\text{reproduce})$ over $R=4$ payloads $\times 10$ sampled continuations per cell, with the scrambled and no-payload controls and a bootstrap interval on the knee location N^* (the interpolated $P=\frac{1}{2}$ crossing).

p	knee N^* (95% CI)	width	scrambled P @ $N=32$	no-payload P
1	7.2 (5.5–10.3)	7.8	0.97	0.00
2	4.3 (1.7–6.2)	5.2	0.70	0.00
3	3.0 (2.6–3.6)	1.6	0.05	0.00
5	1.9 (1.7–4.8)	3.2	0.00	0.00

Table 3: E1.1 on GPT-2-small. The knee N^* decreases with length; the no-payload control is 0 throughout; the scrambled control is near 0 for $p \geq 3$ but *not* for short p (see text).

C1 (the knee exists): confirmed. Every length shows a clear threshold in N (Figure 1), and the no-payload control is 0.00 everywhere (Table 3)—the payloads are genuinely out-of-distribution, so reproduction is caused by the repetitions, not the model’s prior.

C3 (length robustness): confirmed, and stronger than “flat.” N^* *decreases* with length, $7.2 \rightarrow 4.3 \rightarrow 3.0 \rightarrow 1.9$ (Figure 2): longer strings are *easier* to plant. This is the induction-like signature—the opposite of the fixed-order count cache’s doubly-exponential brittleness ladder [2].

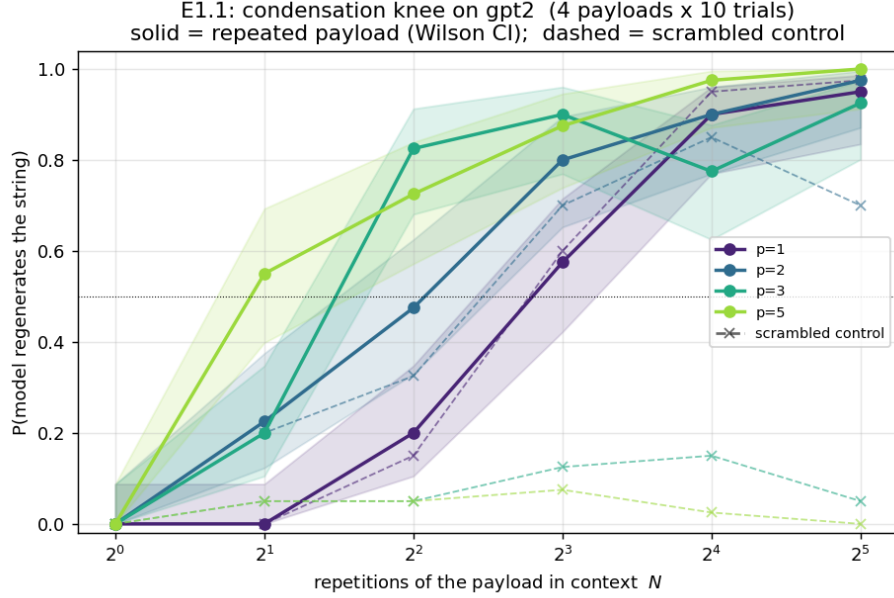


Figure 1: $P(\text{reproduce})$ vs. repetition count N on GPT-2-small, per string length (solid, with Wilson bands); dashed is the scrambled control. A clear knee in N exists for every length. Reproduced by `experiments/e1_condensation_knee.py`.

The scrambled control exposed a length-dependent mechanism. For $p \geq 3$ the scrambled control stays near 0 while the payload locks: reproduction needs the *pattern*. But for $p \leq 2$ the scrambled control reproduces almost as well (0.97, 0.70), because it is *degenerate* at short length—a single repeated token has no order to scramble, and a bigram re-forms by chance. This motivated E1.2, which separates copying from frequency directly.

5 E1.2: copy versus frequency

Is reproduction genuine copy-on-match (induction) or a frequency bias (the repeated tokens simply become likely)? We separate them behaviorally—no activation access. After streaming S a fixed above-knee number of times N , we prime generation two ways: a *self-cue* at S 's own tail (a matching cue), and a *neutral-cue* appending a fresh token $z \notin S$ (a non-matching cue), and compare $P(\text{reproduce } S)$. Induction is cue-triggered (the matching cue reproduces, the non-matching one should not); a frequency bias is cue-independent. The signature is the gap $P_{\text{self}} - P_{\text{neu}}$ vs. length.

p	P_{self}	P_{neu}	gap ($N=16$)	gap ($N=32$)
1	0.78–0.92	0.26–0.29	+0.51	+0.62
2	0.92–0.97	0.07–0.12	+0.79	+0.90
3	0.97–1.00	0.07–0.19	+0.90	+0.81
5	1.00	0.17–0.32	+0.83	+0.68
8	0.99–1.00	0.15–0.26	+0.85	+0.72

Table 4: E1.2 on GPT-2-small ($R=6$ payloads $\times 12$ trials). The matching cue locks; the non-matching cue mostly fails. The gap is large at every length and smallest at $p=1$.

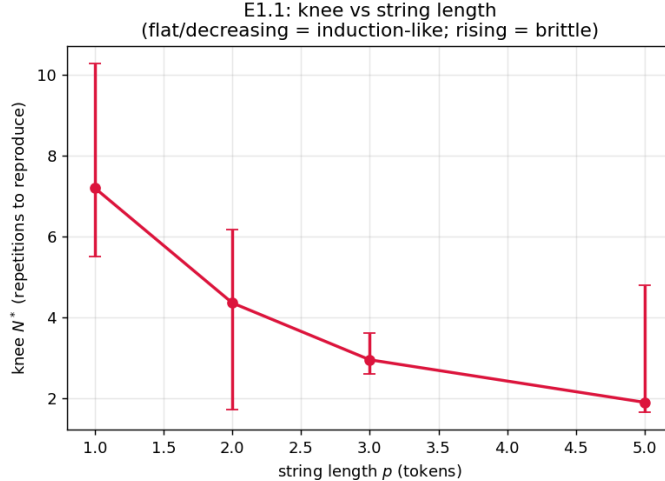


Figure 2: Knee N^* vs. string length (bootstrap CI). Decreasing N^* is the induction-like, length-robust behavior; a rising curve would indicate fixed-order brittleness.

Reproduction is predominantly cue-dependent at every length (Table 4, Figure 3): the non-matching cue mostly fails ($P_{\text{neu}} \leq \sim 0.3$) while the matching cue locks ($P_{\text{self}} \approx 0.8\text{--}1.0$), a gap of 0.5–0.9 well above zero. This is a behavioral induction [3] (copy-on-match) signature, established without interpretability tooling and consistent with C2.

This revises the E1.1 reading. Even $p=1$ is mostly copy (gap +0.5–0.6), not the pure-frequency regime E1.1’s degenerate control suggested; it merely carries the *largest frequency tail* ($P_{\text{neu}} \approx 0.27$, versus $\sim 0.1\text{--}0.2$ for longer strings), since a single token has the biggest cue-independent pull and more distinct tokens dilute any one token’s frequency. The length axis is therefore *copy-dominant-everywhere with a frequency tail largest at $p=1$* , not a clean regime switch.

Remark 1 (P_{neu} over-states frequency). After the non-matching cue the model can still re-enter S if it emits an S -token by chance and induction then completes the pattern (the toy’s re-entry / self-healing). So P_{neu} is an *upper* bound on the pure-frequency component, and the true copy fraction is if anything larger than the gap suggests.

6 E1.5: is the knee an N effect or a context-length effect?

E1.1’s context is $S \times N$, so the number of repetitions and the total context length grow together. E1.5 disentangles them. *Test 1* compares a *growing* context ($S \times N$) against a *fixed* total length (filler padded to a constant length) at each N . The two curves largely coincide (Figure 4, left): at fixed total length the threshold still rises sharply with N , so **the knee is predominantly an N effect**, not an artifact of longer context. A secondary *dilution* effect remains (padding at low N modestly lowers reproduction, strongest for longer p). *Test 2* holds N above the knee and inserts a gap of G neutral tokens between the payload and a re-presented matching cue. Reproduction **decays with distance**—gone by $G \approx 16\text{--}32$ tokens for $p \geq 3$ (Figure 4, right). So in GPT-2-small the copy is recency-modulated, not arbitrarily long-range; a payload buried deep in a long context is much weaker than one near the end. (The gap is filled with rare tokens, so part of the decay may be filler competition rather than pure distance; larger models with longer-range induction should

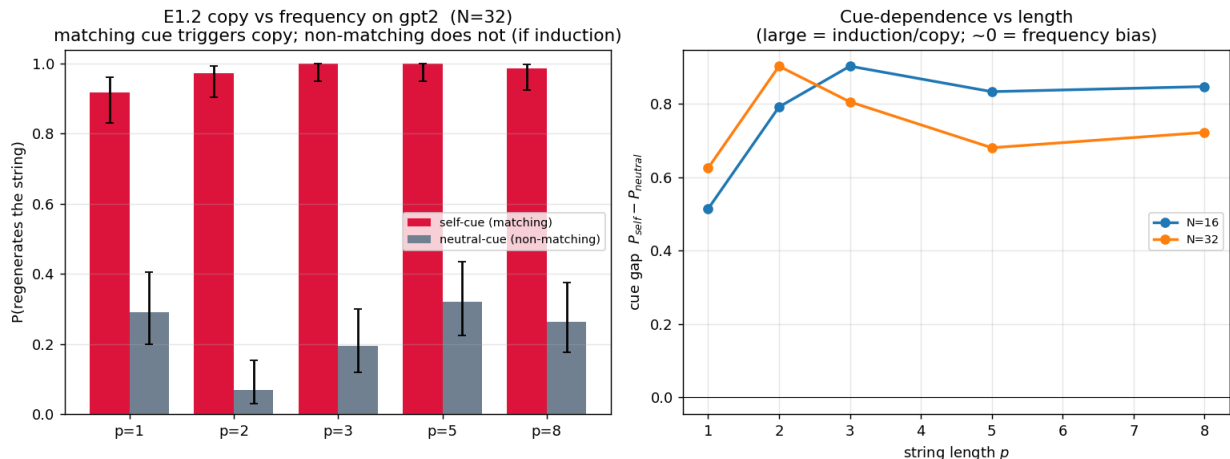


Figure 3: Copy vs. frequency. *Left*: matching (self) vs. non-matching (neutral) cue at $N=32$ per length, Wilson CIs. *Right*: the cue gap $P_{\text{self}} - P_{\text{neu}}$ vs. length; large everywhere (cue-triggered copy), smallest at $p=1$ (largest frequency tail). Reproduced by `experiments/e1_2_cue.py`.

push the limit out.)

7 E1.3-lite: cross-family replication, and an 8 GB wall

A size sweep over GPT-2 {small, medium, large} and Pythia {160M, 410M} was intended. The dominant outcome was a *hardware* one: on an 8 GB machine, gpt2-medium (355M) and larger thrash on swap (gpt2-large, 3 GB of weights, ran 47 wall-minutes using ~9 CPU minutes and completed nothing). Only GPT-2-small (124M) and Pythia-160M (162M) run cleanly, so a genuine size-scaling sweep is **deferred to more compute**. What did run shows the condensation knee and its length-robustness (N^* decreasing with p) in *both* model families (Figure 5), a modest cross-family generality check; the GPT-2 knees also replicate E1.1. Pythia-160M does not lock a single token within $N \leq 32$ (no knee at $p=1$), i.e. it is harder to frequency-drive than GPT-2, consistent with E1.2. The “knee vs. parameters” panel spans only two near-identical sizes and is *not* a scaling result.

8 E2: induction-head attribution

E1.2 showed reproduction is cue-triggered copy (behavioral induction); E2 asks whether the model’s actual induction heads carry it. Run on CPU (TransformerLens warns MPS may be silently incorrect on torch 2.8). First, the standard repeated-random-sequence induction score flags the top heads as **L5H5, L7H10, L6H9, L5H1, L7H2** (then L9H6, L10H1, L9H9)—exactly the canonical GPT-2-small induction heads, validating the detector. Then a causal test: build $S \times N$, measure $P(\text{correct next payload token})$, and ablate the 8 induction heads versus 8 random heads, sweeping N .

The clean signal lives *near the knee*, not at saturation: at $N=16$ the copy is over-determined (many repetitions, redundant heads) and robust to losing 8 heads, but near threshold the circuit is load-bearing. Ablating the induction heads then **collapses** reproduction—e.g. at $N=2$, $p=3$: $0.40 \rightarrow 0.05$ (−87%); $p=5$: $0.48 \rightarrow 0.09$ (−81%)—far beyond the ~15–25% general degradation from ablating random heads (Figure 6). This is the **mechanistic confirmation of C2**: the

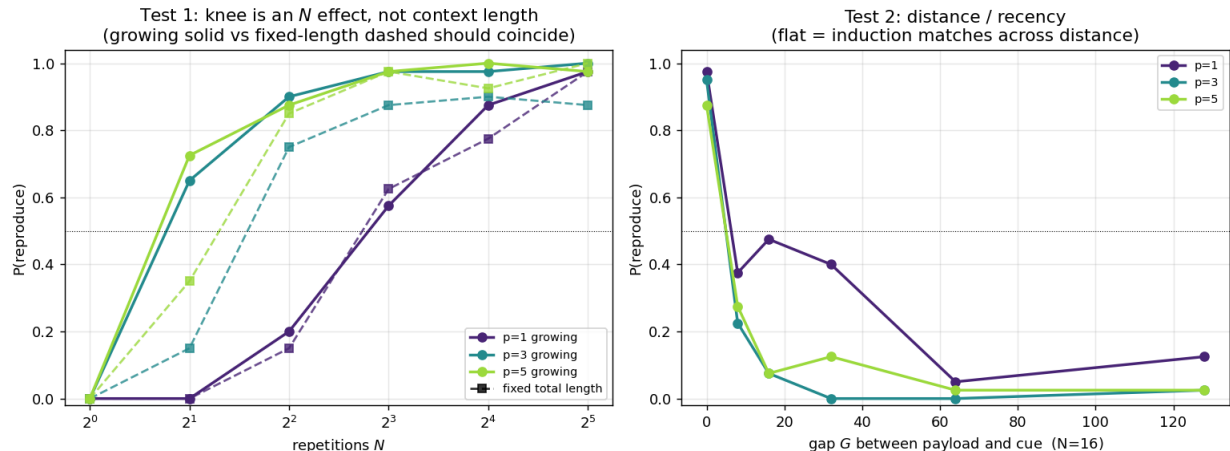


Figure 4: E1.5. *Left*: growing (solid) vs. fixed total length (dashed) – the knee persists at fixed length, so it is an N effect. *Right*: reproduction decays as the payload is separated from the cue by a gap of G neutral tokens (recency-modulated). Reproduced by `experiments/e1_5_context.py`.

real-transformer condensation knee is an induction-head phenomenon. (Caveats: random-head ablation is not exactly zero, so the load-bearing signal is the induction-vs-random *differential*; 8 heads is a subset of the circuit; GPT-2-small only.)

9 E4-lite: poisonability tracks trust, not usefulness

The toy’s headline (C4) is that exploitability follows how much the model *trusts* a span, not how *useful* it is. We test the dissociation with a 2×2 of contexts— {repetitive, non-repetitive} $\times$ {useful (real text), useless (OOD gibberish)}—each scored on three axes: usefulness U (naturalness = mean log-prob of one content unit), trust T (induction-head engagement: attention the E2 heads pay to a repeated bigram’s continuation), and poisonability P (copy-rate of a greedy continuation).

context type	U (log-prob)	T (induction)	P (copy-rate)
repetitive + useful (real sentence $\times k$)	−3.5	0.34	0.77
repetitive + useless (gibberish $\times k$)	−8.7	0.12	1.00
non-repetitive + useful (real prose)	−4.2	0.00	0.00
non-repetitive + useless (random tokens)	−8.3	0.00	0.22

Table 5: E4-lite on GPT-2-small. Poisonability follows the repetition/trust axis, not usefulness: $\text{corr}(P, U) = -0.20$, $\text{corr}(P, T) = +0.54$.

The dissociation is clean (Table 5, Fig. 7): **repetitive gibberish—the least useful content—is fully poisonable ($P=1.0$)**, while **non-repetitive real prose—useful—is not poisonable at all ($P=0.0$)**. This is the real-transformer analogue of the toy’s central finding and its defensive corollary: the quantity to bound is *trust on repeated/templated content*, independent of whether that content is useful. (On a base model “usefulness” is a naturalness proxy, not a downstream task; the instruction-tuned version is deferred to a larger machine.)

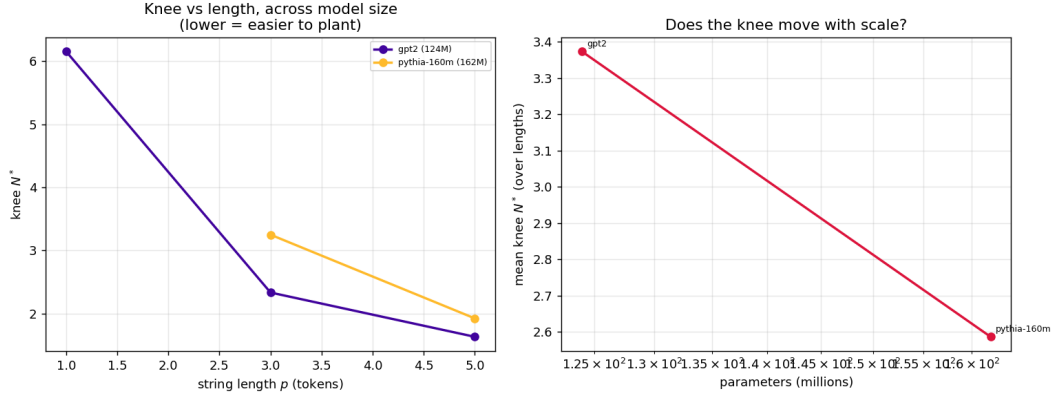


Figure 5: E1.3-lite. *Left*: the knee decreases with length in both GPT-2 and Pythia (cross-family replication). *Right*: two near-identical sizes – not a scaling result; a real sweep needs >8 GB. Reproduced by `experiments/e1_3_size_sweep.py`.

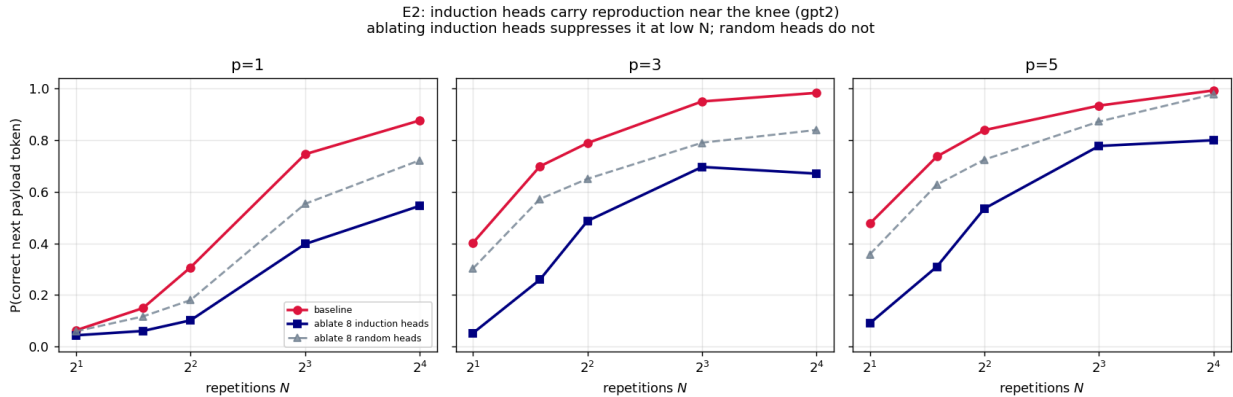
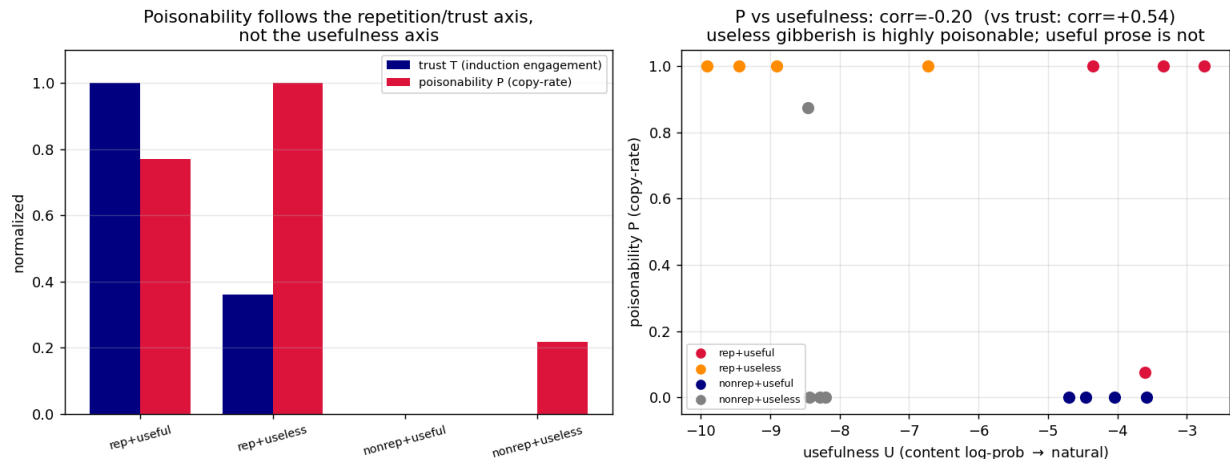


Figure 6: E2. Per length, $P(\text{correct next token})$ vs. N for baseline, ablating 8 induction heads, and ablating 8 random heads. Induction ablation suppresses reproduction far below baseline and below random ablation, most strongly near the knee (low N). Reproduced by `experiments/e2_induction.py`.

10 E5-lite: delivery from inside a document

A scaled-down RAG attack (C5): a coherent prose document (~ 65 tokens) has an OOD payload ($p=3$) repeated k times planted in it; we generate a continuation and measure $\text{hijack} = \text{payload occupancy of the output}$. Two sweeps (Fig. 8). *Entry* (placement, $k=8$): the payload hijacks *only* when it sits at the very end ($\text{frac}=1.0 \rightarrow 0.52$); buried anywhere earlier it is ≈ 0 , even with just ~ 16 tokens of prose after it—sharper than E1.5’s filler gap, because coherent prose competes harder for the continuation. *Lock-in* (repetitions, at the end): a clean knee crossing $\frac{1}{2}$ around $k \approx 10\text{--}12$, *higher* than the bare-context knee (~ 3): embedding in legitimate prose raises the poison cost.

C5 confirmed: $\text{hijack} = \text{entry} \times \text{lock-in}$. Both must hold—the payload must be recent *and* repeated enough to overcome the legitimate content. The danger zone in a long context is the span near where generation happens; burying a payload, or strong surrounding context, suppresses the hijack.



11 E6: the context-window worm

Transmission (C6): an infected host’s output (containing the payload) becomes the next host’s context, passed as *text* (decoded then re-tokenized, as agents and RAG pipelines pass it); we count payload copies in each host’s output across hops.

A real worm with a critical length. In same-model serial passage (Fig. 9, left), a length-1 string *propagates indefinitely* at an endemic load (~ 12 copies/host) while longer strings *die out*, faster as p grows (gone by hop 2 at $p=8$): the basic reproduction number falls with string length and crosses 1 around $p \approx 1-2$, exactly the toy’s prediction (short = epidemic, long = one-shot). The epidemic is carried by *super-spreader* payloads that fully take over a host’s output (occupancy $\rightarrow 1$, ending mid-payload so the next host re-enters); most payloads reproduce weakly and die, tying the critical length to the recency limit of E1.5. **Cross-tokenizer firebreak** (Fig. 9, right): same-tokenizer transmission survives to hop 6, but switching hosts to Pythia-160M collapses it after one hop—re-tokenizing fragments the payload so induction cannot copy it cleanly. A heterogeneous-model pipeline is naturally worm-resistant.

12 The paired detector

The attacks of §9–11 all key on one quantity: the induction trust a repeated span earns. The defense watches the same quantity. We score a context by its mean *induction engagement*—the attention the canonical induction heads (from E2) pay to repeated-bigram continuations—which is high on a repeated/poisoned span and ≈ 0 on natural prose, regardless of whether the content is meaningful. As a binary detector (clean prose documents vs. the same documents with an OOD payload repeated k times), it separates the two *perfectly*: ROC-AUC = 1.0 at every $k \in \{2, 4, 8, 16\}$, with the score rising monotonically in k (Fig. 10). It fires already at $k=2$, well below the in-document hijack knee (~ 10 –12, §10): it flags the poisonable configuration *before* it is dangerous, and on the trust signal independent of usefulness.

Flagging—or capping the trust earned by—repeated spans that approach the condensation knee is precisely what neutralizes the E5 hijack and the E6 worm; it is the transformer realization of

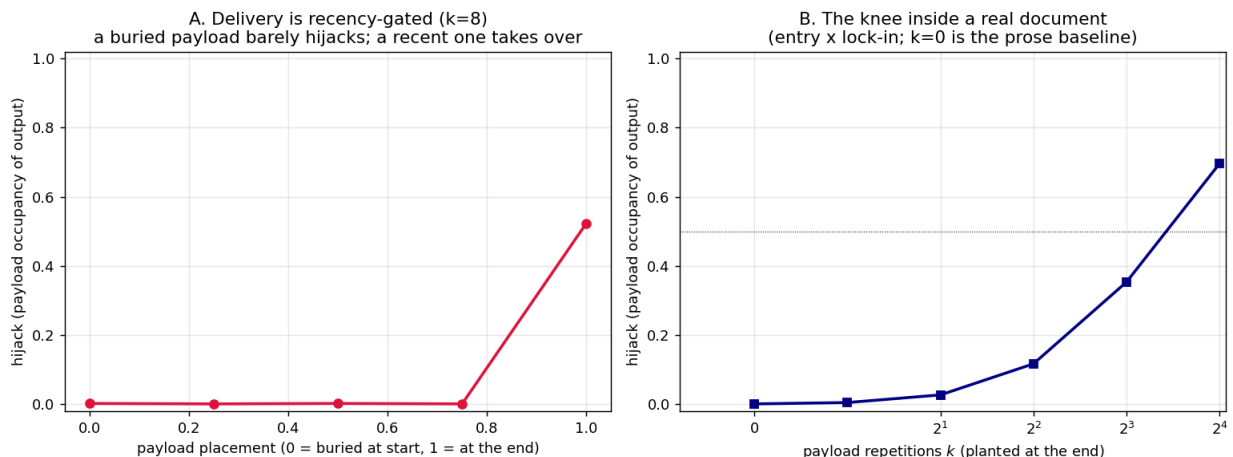


Figure 8: E5-lite. *Left*: hijack vs. placement—delivery is recency-gated, only a payload at the end takes over. *Right*: hijack vs. repetitions at the end—the knee inside a real document. Reproduced by `experiments/e5_hijack.py`.

the toy’s defensive corollary, *bound trust, not usefulness*. *Caveat*: the detector flags *any* induction-engaging repeated span, malicious or benign (boilerplate, templates, repeated instructions). That is the correct behavior under the thesis—such spans are genuinely poisonable (§9)—but it identifies the poisonable *configuration*, not malicious *intent*, so a deployment needs a benign-repetition allowlist or a trust cap rather than a hard block.

13 Threats to validity

Beyond the small scale (one model, $R \sim 4\text{--}6$ payloads, $\sim 72\text{--}128$ samples per cell, sampled $T=1.0$ only), the standing concerns are: **tokenization**—length is in tokens and OOD spans tokenize unstably, addressed by a round-trip-stability filter; **decoding**—the knee persists under *greedy* decoding (with lower N^* , less sampling noise to break reproduction), so it is a model property, not a sampling artifact; a full temperature sweep is still future work; **instruction-tuned models** may comment or refuse rather than copy, so base models are used for the clean mechanism and chat models are a separate condition ([**TBD**]); **context-length vs. N** —addressed by E1.5 (§6): the knee persists at fixed total length, so it is an N effect, though reproduction is distance-sensitive; and **scale**—induction strengthens with model size, so the knee is expected to move, but the size sweep is capped at 162M by the 8 GB machine (§7) and needs more compute.

14 What is established, and what is not

On GPT-2-small, with controls and confidence intervals: a repeated out-of-distribution string is regenerated past a sharp condensation knee (C1); the knee *decreases* with string length, the induction-like length-robustness predicted by the toy (C3); the knee is a repetition effect (not context length) though distance-sensitive (E1.5); reproduction is a cue-triggered copy rather than a frequency bias (C2, behaviorally); and—the new result—ablating the canonical GPT-2 induction heads *causally* collapses reproduction near the knee, far beyond random-head damage, so **C2 is mechanistically established** on this model (E2). The downstream claims also hold on GPT-

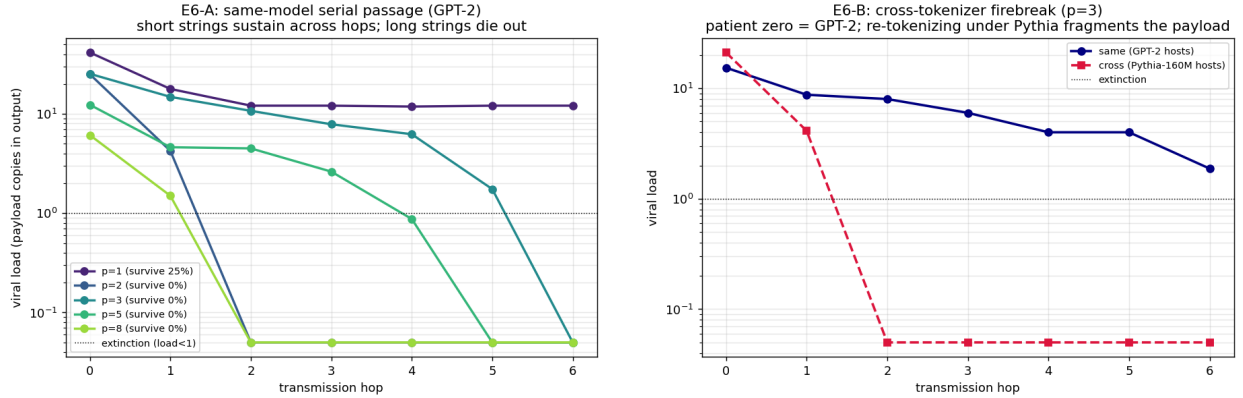


Figure 9: E6. *Left*: same-model serial passage—short strings ($p=1$) sustain at an endemic load while longer strings die out, faster as p grows (a critical length). *Right*: the cross-tokenizer firebreak—GPT-2→GPT-2 transmission persists, GPT-2→Pythia collapses after one hop. Reproduced by `experiments/e6_worm.py`.

2-small: poisonability is dissociated from usefulness and aligned with induction trust (C4, §9); a planted span hijacks generation as entry $\times$ lock-in (C5, §10); and the worm has a critical string length and a cross-tokenizer firebreak (C6, §11). The phenomena replicate in a second model family (Pythia-160M). So all six conjectures C1–C6 have a qualitative real-transformer result, and a paired detector (§12) realizes the toy’s bound-trust defense (poisoned vs. clean documents separate at AUC 1.0 on the induction-trust signal). What is *not* yet established is **scale**: how the knee, the worm’s critical length, and the induction range move with model size (the sweep is capped at 162M by 8 GB), and whether instruction-tuned models behave the same—plus the larger-context, retrieval-grounded versions of C5 and C6, and a real-task (not naturalness-proxy) usefulness for C4. Those need more than an 8 GB machine and are the next stage.

References

- [1] D. Shumow. *Trust Is the Attack Surface: A Cryptanalytic Reading of In-Context Memory Poisoning in a Markov Mixture Model*. Project `trust-is-the-attack-surface`, 2026.
- [2] D. Shumow. *Contagious Context: Self-Reproducing and Transmissible Token Strings in a Markov Cache*. Project `trust-is-the-attack-surface`, 2026.
- [3] C. Olsson et al. *In-context Learning and Induction Heads*. Transformer Circuits, 2022.
- [4] A. Bietti, V. Cabannes, D. Bouchacourt, H. Jégou, L. Bottou. *Birth of a Transformer: A Memory Viewpoint*. NeurIPS, 2023.
- [5] S. Cohen, R. Bitton, B. Nassi. *Here Comes the AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications*. 2024.
- [6] K. Greshake et al. *Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. AISec, 2023.
- [7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019.

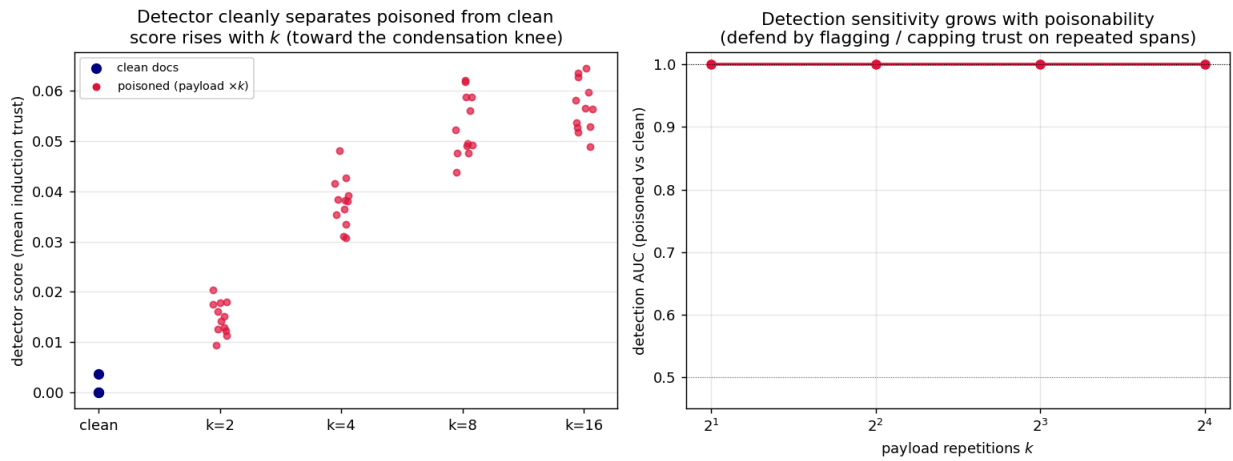


Figure 10: The detector. *Left*: score (mean induction trust) for clean docs vs. docs with a payload repeated k times—clean sits at ≈ 0 , poisoned rises with k . *Right*: detection AUC vs. k ($= 1.0$ throughout). Reproduced by `experiments/e7_detector.py`.